

# **Software Construction and Development - Assignment 1**

## **Phase 2: Problem Definition and Requirements**

**Submitted by: Laiba Siddiqui (24K-3028) and Laiba (24K-3075)**

### **1. Architecture Overview**

ExpenseMate is designed as a personal expense management system. Its features include authentication and OTP flows, transactions and categories, budgets, savings goals, dashboard summaries, and CSV/report functions. The architecture is designed in a way that it must keep the user interface independent from persistence and business rules.

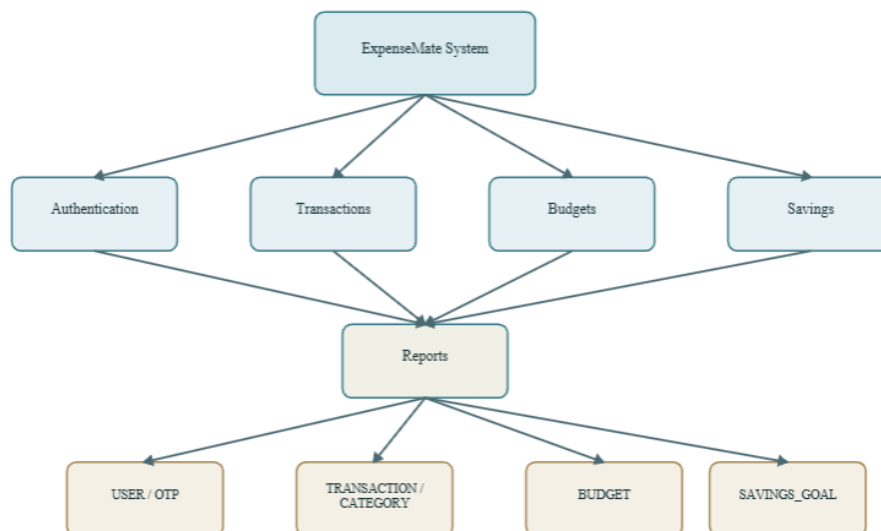
**Selected architectural style: Client–Server.** The client is responsible for presentation and user interaction. The server-application layer handles authentication, validation, business rules, reporting, and repository access. A database server stores persistent data. This ensures a smooth work flow of the entire project.

### **2. Why Client–Server fits the project**

The Client-Server architecture separates screens from database logic, while services handle authentication, validation, budgets, savings, and reports. It improves security by keeping database credentials on the server, supports future web or mobile clients, and makes the system easier to maintain by separating database operations from business rules.

### **3. Logical View**

The logical view describes the major functional responsibilities and the conceptual data they operate on. It focuses on what the system does, rather than where the code runs.



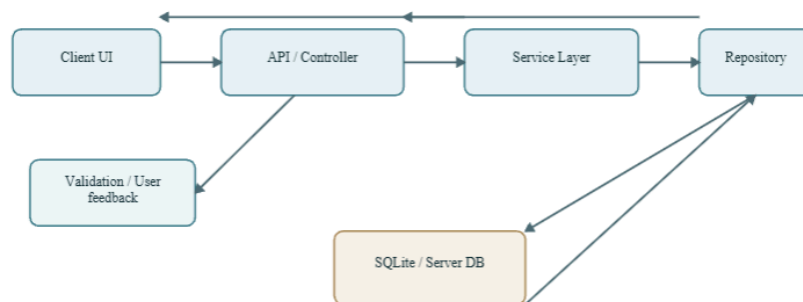
### **4. Logical components**

| Module | Main responsibilities |
|--------|-----------------------|
|--------|-----------------------|

|                           |                                                                                                                  |
|---------------------------|------------------------------------------------------------------------------------------------------------------|
| Authentication            | Register, login, logout, OTP verification, password reset, password/email updates.                               |
| Transactions & Categories | Create, read, update and delete income/expense transactions; validate amount/date; manage categories.            |
| Budgets                   | Create/update budgets, calculate category spending, budget percentage, remaining amount, suggestions and alerts. |
| Savings                   | Create/update/delete goals, add contributions, calculate remaining amount and progress percentage.               |
| Reports & Dashboard       | Dashboard totals, filtered report transactions, category breakdown, CSV import/export.                           |
| Core entities             | USER, OTP, CATEGORY, TRANSACTION, BUDGET, SAVINGS_GOAL.                                                          |

## **5. Process View**

The process view shows how a request moves at runtime. The intended flow keeps the user interface lightweight while validation and business rules execute in the application/service layer.



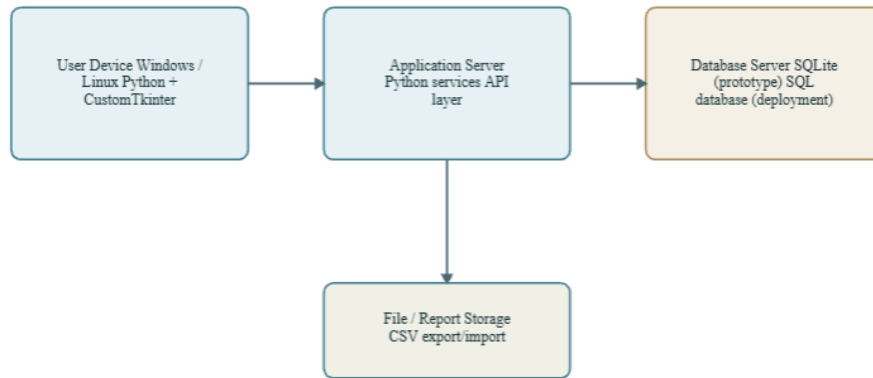
## **6. Example: Add Transaction**

1. The user enters transaction type, amount, category, date and optional note in the Transactions screen.
2. The client calls the backend interface/service function instead of accessing SQLite directly.
3. The service identifies the user and validates transaction type, amount, category and date.
4. The repository creates the transaction record in the database.
5. The service returns a success/message/data result to the client.
6. The screen refreshes the displayed transaction list and gives feedback to the user.

## **7. Physical View**

The physical view maps software responsibilities to the computing environments on which they operate. The current repository is a desktop prototype with SQLite; the construction plan separates the application and database responsibilities so they can be deployed as independent server-side components.

Physical allocation of major software and hardware nodes

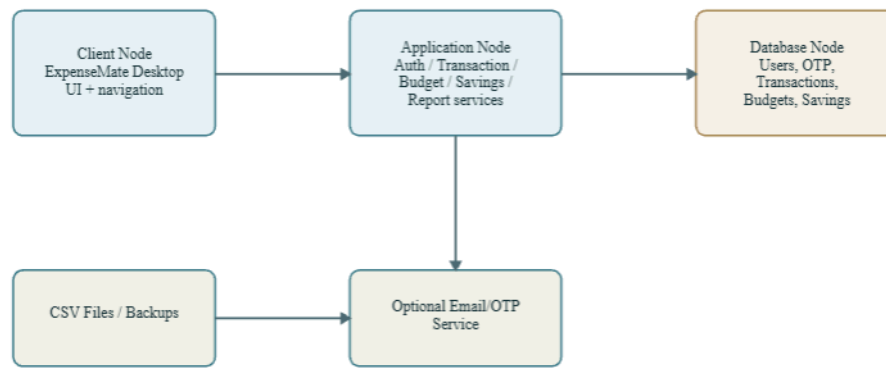


## 8. Physical Nodes

| Node                              | Software / resources                                                               | Purpose                                                                                           |
|-----------------------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| User Device                       | Python runtime,<br>CustomTkinter, Pillow,<br>ExpenseMate client                    | Runs screens, navigation and<br>presentation logic.                                               |
| Application<br>Server             | Python application/service layer;<br>API/controller interface                      | Runs authentication,<br>transactions, budgets, savings<br>and reporting logic.                    |
| Database<br>Server                | SQLite for current prototype;<br>SQL database service for<br>multi-user deployment | Persistent storage and controlled<br>data access.                                                 |
| File/Report<br>Storage            | CSV files / backup location                                                        | Supports transaction import/export<br>and backup workflows.                                       |
| External<br>service<br>(optional) | Email/OTP provider                                                                 | Real OTP delivery in a production<br>deployment; current prototype<br>prints mock OTP to console. |

## 9. Deployment View

The deployment view describes the runtime nodes and communication paths when the system is installed. It is intentionally different from the physical view: the physical view describes the allocation of software/hardware responsibilities, while the deployment view emphasizes deployable nodes and their runtime connections.



## **10. Construction Timeline & Gantt Chart**

The proposed construction plan assumes a two-member team. Member A owns the architecture views and diagrams; Member B owns coding standards, tools and timeline planning. Both members participate in architecture selection and implementation/integration.

## **11. Work breakdown**

| Task                                            | Owner    | W1 | W2 | W3 | W4 | W5 | W6 |
|-------------------------------------------------|----------|----|----|----|----|----|----|
| Architecture & requirements review              | Both     | ■  | ■  |    |    |    |    |
| Logical / Process / Physical / Deployment views | Member 2 | ■  | ■  | ■  |    |    |    |
| Coding standards & tool setup                   | Member 1 | ■  | ■  |    |    |    |    |
| Client UI refinement                            | Both     |    | ■  | ■  |    |    |    |
| Backend services & repositories                 | Both     |    | ■  | ■  | ■  |    |    |
| Database & integration                          | Both     |    |    | ■  | ■  | ■  |    |
| Testing & bug fixing                            | Both     |    |    |    | ■  | ■  | ■  |
| Reports / CSV / final integration               | Both     |    |    |    |    | ■  | ■  |
| Documentation & final review                    | Both     |    |    |    |    | ■  | ■  |